SOURCE-LEVEL GUIDE · OPENDIRECTOR

OpenDirector：从原理到源码

一份给想贡献或定制 OpenDirector 的工程师的导读：这个本地 AI 视频工作台要解决什么问题，用户和 Agent 怎样共用同一个工程，文件存储怎样做到崩溃可恢复，付费生成任务怎样避免重复提交，以及代码该从哪里读起。

kzhou003/OpenDirector @ 4c05ef3

怎么读这份导读

这份导读写给第一次接触 OpenDirector、想**读懂它、给它提 PR**，或者**在它之上定制**的工程师。

OpenDirector 是一个在本机运行、用浏览器操作的 AI 视频创作工作台。它最值得读的地方不在画布界面，而在底下那套规则：**用户点按钮和 Agent 跑命令，走的是同一条业务路径**；工程是一串不可变的文件提交，崩溃后能接着写完；一次付费的视频生成，无论网络怎么断，都不会被静默地再提交一次。

全文按这个顺序展开：

1. **全景**：要解决的问题、系统边界、要求/目标/限制表。
2. **启动与运行形态**：一个 Python 包怎样带上前端和 Agent 程序，数据放在哪里。
3. **业务模型**：核心对象，以及“参考、实际来源、当前采用”三种关系。
4. **命令与存储**：所有写入的唯一入口，提交日志怎样保证崩溃可恢复。
5. **生成任务**：从提交到采用，以及恢复决策表。
6. **模型接入**：能力规则和协议适配器。
7. **工具任务**：抠图、重绘这类“配方”为什么有自己的台账。
8. **Agent 运行时**：怎样把 Codex、Claude Code 这些外部程序接进来。
9. **前端**：画布是投影，同步靠事件游标。
10. **时间线与导出**：为什么 MP4 只能在浏览器里合成。
11. **端到端生命周期**：一条完整的 Agent 请求。
12. **开源许可证**，以及一段不能略过的项目早期历史。
13. **贡献与定制**：开发命令、review 规则、入手方向。

> **要点 · 固定版本与范围**
>
> - 代码：`kzhou003/OpenDirector` @ `4c05ef3`（2026-11-06）。这是 `ThinkFlowLab/OpenDirector` 的 fork，该 commit 在上游也存在。
> - 仓库有 710 个受跟踪文件，其中测试 224 个。版本号 `0.1.0`，首次公开发布就在这几天，接口仍在快速变化。
> - 文档与代码的关系：项目自己说明“架构与路线描述目标，不能据此推断某项能力已经可用”，见 `docs/README.md`。本文的事实以代码为准。
> - 标注“实测”的内容，是我在这个 commit 上实际运行过的；标注“推断”的内容，是读代码得出、没有运行验证的。

全景：问题、目标、限制

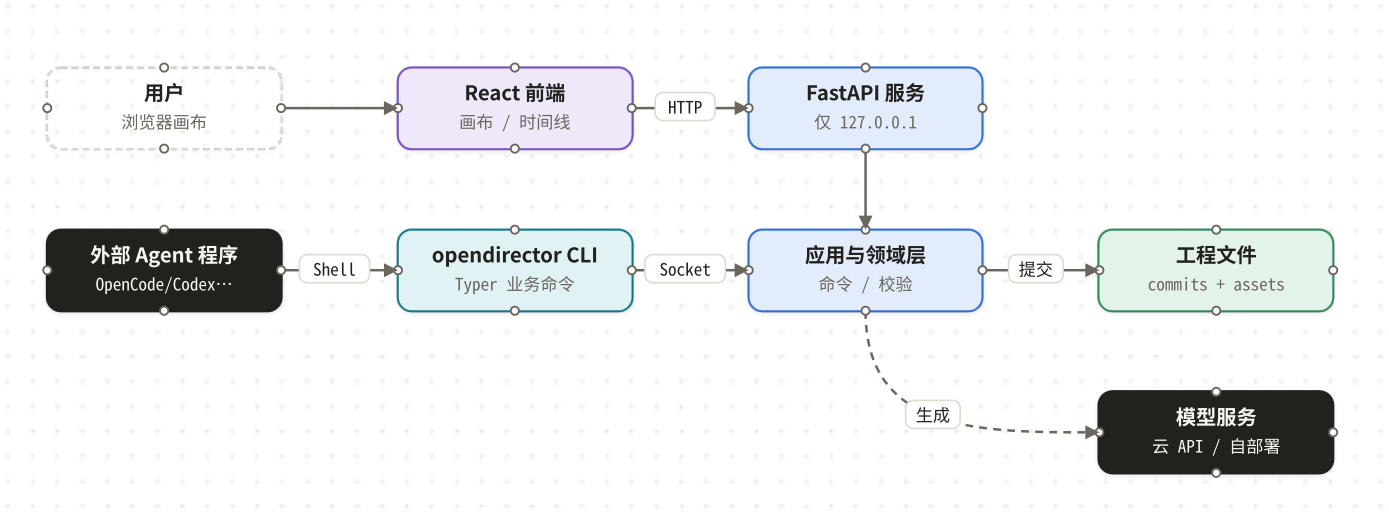
没有它时有什么问题

做一支 AI 短片要串起好几种工具：和大模型聊脚本、在 A 服务出图、在 B 服务出视频、在剪辑软件里拼接。麻烦不只是切换窗口：

环节	问题	OpenDirector 的回答
创作过程	提示词、参考图、生成结果分散各处，过两天说不清某个镜头是用什么做出来的	工程保存生成配置、参考关系和每次执行的固定输入
模型调用	视频生成又慢又贵，网络一断就不知道任务到底提交成功没有，重试可能多付一次钱	任务先落盘再提交，未知状态绝不自动重发
Agent 协作	通用 Agent 只能产出文件，用户拿到后接不上；用户改过的东西 Agent 也看不到	Agent 通过业务命令操作同一个工程，产物就是可继续编辑的节点
模型选择	每家服务的参数、限制、协议都不同，而且很多人想用自部署模型	按服务声明能力，后端统一校验，用协议适配器接入

系统边界

系统全景：两个入口，一个工程



这张图的重点是中间那一列：前端走 HTTP，Agent 走 CLI，但都汇入同一组应用服务。深色节点不在本仓库里，OpenCode 程序是构建时从 npm 取得的二进制。入口见 `create_app` 和 `cli.Client`。

仓库里没有的东西：模型本身和云服务；Agent 程序（内置的 OpenCode 是 npm 依赖，`opencode-ai 1.19.5`；Codex、Claude Code 等需要用户自己安装和登录）；FFmpeg/ffprobe。仓库也没有用户账号、云同步或多人协作，因为它就是单机应用。

`tools/fasth3/` 是一套在 GPU 云主机上部署自托管视频模型的脚本，和主程序分开，见 `README.txt`。

要求 / 目标 / 限制

这张表是全文的骨架，后面每一章都在解释其中一行。

项目	内容	在哪一章解决
核心目标	用户与 Agent 在相同授权下能力等价，共同操作一个工程（`AGENTS.md`）	命令与存储、Agent 运行时
功能要求	素材管理、多模态生成、媒体编辑、候选采用、时间线、导出 MP4/JSON/FCPXML	业务模型、生成任务、工具任务、导出
成本限制	模型调用要花钱，且远端状态不可回滚，所以“未知提交不能自动重发”	生成任务
可靠性限制	进程随时可能被杀，写入中断后不能损坏工程，也不能丢回执	命令与存储
规模压力	单机、单工程单窗口编辑；媒体文件大，历史长；要防止持锁读历史拖垮保存	命令与存储
外部限制	各家模型的参数和字节限制不同；浏览器没有 GPU 时 H.264 编码能力受限	模型接入、导出
架构取舍	模块化单体，不引入微服务、消息队列或数据库做业务存储（`AGENTS.md`）	全文
国际化	界面、错误、目录显示名都要中英双语，业务值不翻译	贡献与定制
许可证	Apache-2.0，随包分发 MPL-2.0 的 `mediabunny` 等组件	开源许可证

启动、打包与运行形态

职责：让用户 `pip install -e .` 之后，用一条 `opendirector serve` 就拿到完整应用。

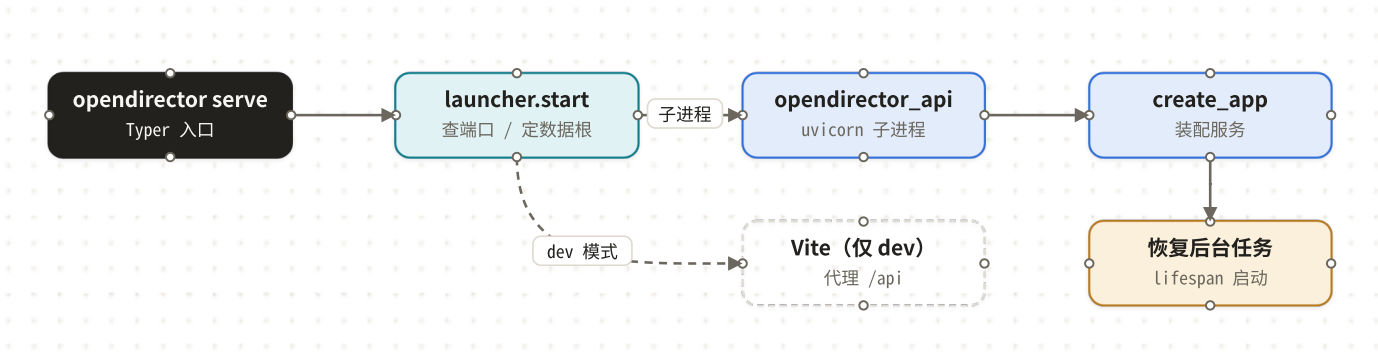
难点：应用横跨 Python 和 Node，还要带一个平台相关的 Agent 二进制，但用户不应该手动跑 npm。

一个 Python 包带上所有东西

项目用自定义的 PEP 517 构建后端解决这件事。`prepare_resources` 依次运行 `npm ci`、`npm run build`，校验 OpenCode 能执行 `--version`，再把前端产物、内置技能、OpenCode 二进制和它的许可证复制到 `_bundled/`。因为带了原生二进制，`PlatformWheel` 把 wheel 标成 `py3-none-<平台>`，不能跨平台使用。

源码职责仍然分开：领域和应用逻辑在 `packages/core/src/opendirector`，HTTP 层在 `apps/api/src/opendirector_api`，但只发一个包，见 `setup.py`。

启动路径：CLI 拉起 API，再等健康检查



启动顺序见 `launcher.start`：先检查端口，避免子进程起来后才报错；API 健康检查通过才启动 Vite。服务启动时 `CreationGeneration.start` 会扫描所有工程，恢复未结束的任务。

三种数据根

安装方式	前端来源	默认数据根	依据
editable（`pip install -e .`）	checkout 下的 `dist/`	**checkout 目录本身**	`data_root`
wheel	包内 `_bundled/web`	`~/.opendirector/`	`web_root`
任意	不变	`OPENDIRECTOR_ROOT` 指定	`packaging.md`

数据根下有两个目录。业务内容放在 `projects/<工程名>/`，运行协调和凭证放在 `.opendirector/`：

```
projects/<工程名>/
  project.json            指向当前 head 的清单，发布它就是提交点
  commits/<commandId>.json 每次提交一份，含完整快照、回执和事件
  assets/<assetId>        不可变素材文件
```

```
  tool-tasks/<id>.json    工具任务台账（不进提交链）
  agent-runs/             对话、每轮工作目录、原生轨迹
  views/<clientId>.json   每个客户端的视口等界面状态
  .generation-results/    模型结果的本地缓存
.opendirector/
  catalog.lock            全局文件锁
  catalog.sqlite3         只存编辑租约
  creation-commits/       待恢复的提交日志
  creation-uploads/       上传暂存区
  models.json             模型配置，含明文 API Key
```

只对本机开放

API 没有用户认证，靠三道检查把自己限制在本机，见 `local_requests`：`Host` 必须是本机地址（挡 DNS rebinding）；`Origin` 如果存在必须是本机（挡别的网站）；必须带自定义头 `X-OpenDirector-Request: 1`。服务也只绑定 `127.0.0.1`，见 `server.main`。

> **注意 · 容易误解：不要直接暴露到局域网**
>
> 上面这些检查不等于认证。把服务放到反向代理后面对外开放，API Key、素材和本机文件访问能力都没有其他保护。模型密钥以明文存放（POSIX 下文件 `0600`），接口不返回密钥，这是项目明确写下的取舍，见 `validation.md`。

业务模型：对象与三种关系

职责：定义工程里有什么。这一层只有 Pydantic 契约和校验，不碰文件、网络和界面。

难点：生成式创作里，“我想要什么”“实际跑了什么”“现在用的是哪个”很容易混成一个字段，而混了之后，重新编辑就会改写历史。

一个工程就是一个快照

`ProjectSnapshot` 是整个工程的聚合根，所有对象都在这一个结构里。

对象	作用	关键字段	代码
Asset	不可变的素材文件和媒体事实	`fingerprint`、`metadata`、`provenance`	`Asset`
GenerationSpec	下一次要生成什么，可编辑	`prompt`、`modelConfigId`、`parameters`、`currentResultId`	`GenerationSpecBase`
Reference	某个配置用哪个来源、什么角色	`targetSpecId`、`role`、`binding`	`ReferenceBase`
CanvasItem	画布上的展示对象	`position`、`binding`、`visible`、`locked`	`CanvasItemBase`
EditorDocument	时间线、图层、主体等可重开的结构	`kind`、`data`	`EditorDocumentBase`
Job	一次执行的固定事实	`inputSnapshot`、`externalTaskId`、`status`、`recovery`	`Job`
ResultVersion	一个候选结果	`specId`、`jobId`、`asset`	`ResultVersion`

校验不在业务代码里零散分布。`project_relations` 在每次解析快照时检查全图一致性：对象 ID 唯一、不跨工程、每个 `AssetRef` 的指纹都和素材一致、编组无环。注意这里**允许**参考关系成环，因为参考只读取当前输出，不会递归执行上游。

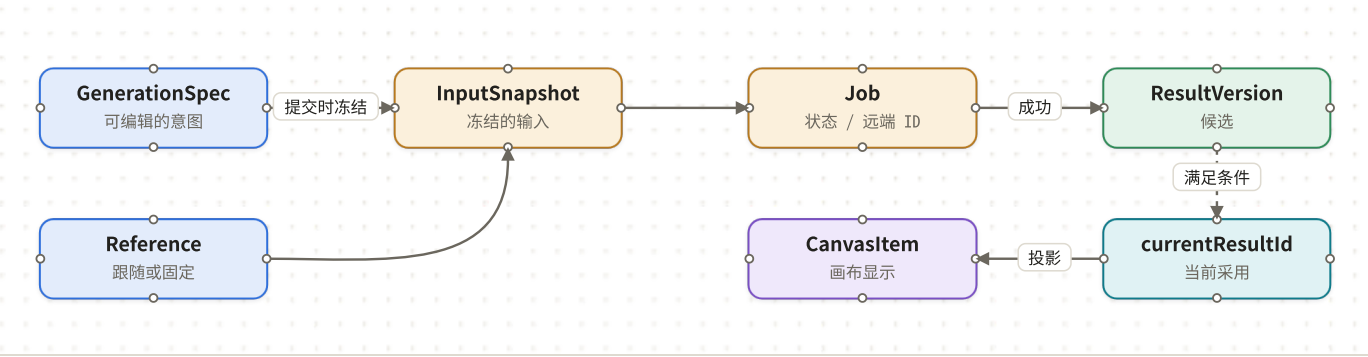
三种关系

这是理解这个项目最重要的一节。`架构文档` 把三种关系分得很清楚：

类比 · 采购单、寄出的订单、货架

把生成想成采购。**GenerationSpec** 是桌上那张还能涂改的采购单草稿。点下提交，系统复印一份寄出去，这份副本就是 **Job 的 InputSnapshot**，之后不管草稿怎么改，寄出的订单都不会变。货到了是 **ResultVersion**，它只是仓库里多了一件货。摆到货架上给人看的那件才是 **currentResultId**。到货不等于上架：如果你在等货时已经换了别的商品上架，新到的货只进仓库，不会把货架上的换掉。

意图、执行、采用是三个分开的对象



左边可以随时改，右边是事实，中间那一步是单向的。`InputSnapshot` 的注释写明它只含创作输入，布局、选择和凭证都排除在外，见 `InputSnapshot`。

参考有两种绑定方式：`FollowingReference` 跟随来源节点的当前内容，`PinnedReference` 固定到某个素材版本。提交时 `resolve_reference` 把它们解析成确定的 `ResolvedReference`。解析只走一层，所以循环引用不会造成递归。

节点的身份由 binding 决定

一个画布节点是空白、素材、生成结果还是编辑器，不靠节点类型判断，而靠 `binding.role`，共有五种：`empty`、`material`、`generation`、`editor`、`container`。隐藏节点只是 `visible=false`，素材、配置和历史都还在。

注意 · 容易误解

- **画布连线不是工作流。** 连线表达“生成时参考什么”，不会触发上游执行，也不代表时间线顺序，见 `AGENTS.md`。别按 ComfyUI 的节点图来理解。
- **删掉正文里的引用标签不会断开连线。** 正文引用只在插入时建立关系，断开连线才会移除来源，见 `架构文档`。
- **指纹不等于身份。** 内容相同的两个来源是两个来源，名称和指纹都不能替代逻辑身份。

命令与存储：唯一的写入路径

职责：保证所有修改都经过同一个校验和持久化边界，不管它来自界面、Agent 还是后台任务。

难点：要在普通文件系统上得到接近数据库事务的语义：要么整个提交生效，要么没有生效；崩溃后能继续；重试不重复。项目特意不用数据库存业务数据，这样工程就是一个可以直接查看和备份的目录。

命令是类型化的

所有写入都是 `Command`：`commandId` 提供幂等身份，`expectedVersions` 声明修改基于哪些对象版本。一共 40 种命令类型，登记在 `_COMMANDS` 这张表里。`creation.batch` 可以把最多 500 个操作打包成一次原子提交，见 `BatchPayload`。

回执只有四种状态，见 `CommandReceipt`：

状态	含义
`applied`	已修改工程
`accepted`	已修改工程，并创建了后台任务
`noop`	校验通过，但没有变化
`rejected`	被拒绝，工程没变，但**拒绝本身也会写入提交链**

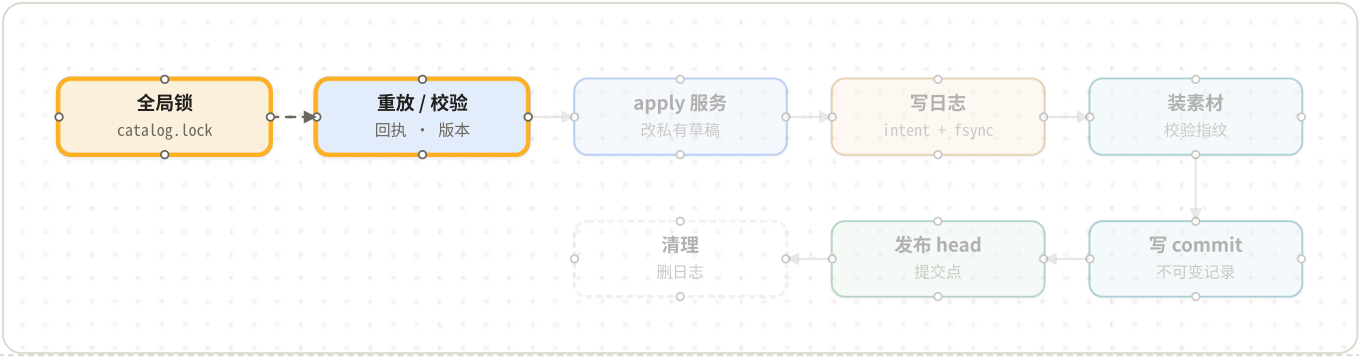
最后一点很特别：被拒绝的命令也有提交记录，这样用同一个 `commandId` 重放时能拿回同一份拒绝回执。

`CreationProjects.execute` 按命令类型分发给各个服务，但它们最终都调用 `commit`。

提交日志

> **类比 · 总账、书签和记账草稿**
>
> `commits/` 是一本只追加的总账，每页都抄了当时的完整账目。`project.json` 是夹在里面的书签，翻到哪页，哪页就是现状。写新一页之前，先在桌上放一张草稿（`creation-commits/` 里的日志），写明这一笔要怎么记。如果中途停电，第二天来人看到草稿，照着写完就行。书签没动之前，一切都算没发生。

▶ 一次提交的步骤（包含崩溃恢复点） 1 / 7



1. 加锁，先看是不是重放

进入 `transaction` 拿到全局文件锁，先完成上次遗留的日志，再用 `_replay` 查提交链：同 ID 同输入返回原回执，同 ID 不同输入报 `command_conflict`。

2. 校验租约和版本

非后台来源必须持有有效租约（`_lease`）。`validate_command` 比较 `expectedVersions` 和当前版本，不一致就是 `revision_conflict`。

3. 在私有副本上修改

业务服务拿到的是快照的副本。改完后整图重新校验，`object_changes` 算出变化，并再次要求：每个被改的对象都必须在 `expectedVersions` 里，版本号只能加一。这条规则也约束内部代码。

4. 写提交日志

`_publish` 把完整的 `CommitIntent` 写入日志并 fsync。这一步之后，这个提交就会被完成。

5. 安装素材，写不可变记录

`install_files` 边复制边算 SHA-256，指纹不符就中止，而且从不覆盖已有文件。然后写 `commits/<commandId>.json`。

6. 发布 head：这才是提交点

`_finish` 原子替换 `project.json`。在此之前崩溃，读者仍然看到旧 head。

7. 清理

删除要删的素材文件，再删除日志。任何一步崩溃，下次调用 `_recover` 都会重新执行 `_finish`，每步都是幂等的。

文件顶部的注释把这个设计说得很完整：“发布 project.json 是提交点”，“SQLite 只协调租约，绝不用来回滚文件写入”，见 `creation_projects.py`。`_checkpoint` 是一个空方法，只用来在测试里注入故障，见 `_checkpoint`。

编辑租约：一个工程一个窗口

`_open` 拒绝第二个 session 打开同一工程。租约 60 秒（`LEASE_SECONDS`），浏览器每 15 秒续约一次（`keepAlive`）。来源为 `background` 和 `external` 的写入不检查租约，所以任务完成时窗口关了也能保存，见 `commit`。

性能怎么撑住

每个提交都存完整快照，读取又要校验提交链，这显然有代价。代码注释记录了项目实际遇到的数字：有几百个提交的工程，每次历史查询曾经要约 13 秒，还持着全局锁，见 `_record`。现在的解法是两级缓存：按文件签名缓存已解析的提交（`4096 条 / 512 MB`），另外单独缓存每个工程的 head（`_current`）。长读取只在锁内确定 head，遍历不可变文件时放开锁（`_read_manifest`）。

> **注意 · 容易误解**
>
> - **全局锁是全局的。** `catalog.lock` 不分工程，超时 15 秒。`apply` 回调运行在锁内，所以不要在里面做网络请求或耗时计算。
> - **磁盘占用随历史增长。** 工程越大，每次提交写的快照就越大，连拖动节点也不例外（推断：拖动结束后才提交，但仍是一次完整快照）。素材文件本身不在快照里。
> - `ViewState` **不走提交链。** 视口存在 `views/`，这样保存视口失败不会影响业务提交。
> - **不做格式迁移。** `formatVersion` 不等于 2 的工程直接拒绝读取，不自动转换，见 `_manifest`。这是有意的规定，见 `AGENTS.md`。

撤销和删除

撤销不是回退到旧提交，而是写一条新提交。`UNDOABLE` 列出可撤销的命令，只有 `ui` 和 `agent` 来源的操作可撤销，后台任务的完成不可撤销。范围也只限 `画布、配置、参考和编辑文档`，素材和 Job 这类事实不会被回退。

删除素材要先走影响预览：`delete_preview` 算出使用情况，命令必须带 `expectedPreview` 指纹，确认后工程变了就要重新预览。

生成任务：不重复花钱

职责：把一个 GenerationSpec 变成模型输出，并按策略采用。

难点：调用模型是有费用的外部副作用，没法放进事务。提交请求发出后网络断了，你不知道对方收到没有。

核心规则：只有能证明没发出去的任务，才能自动提交。

接受：把意图冻结成事实

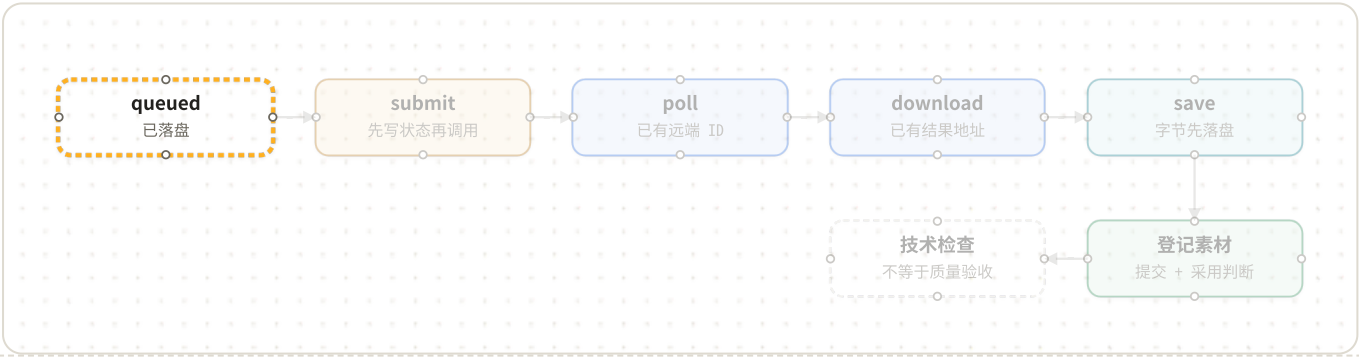
入口是 `CreationGeneration.execute`。`prepare` 在一次提交里完成全部事情：解析参考、校验模型能力、算出实际参数、生成 `InputSnapshot` 和 Job。Job 的 ID 是 `uuid5(commandId, 'job')`，所以同一命令永远对应同一个任务。同一配置最多 8 个排队任务。

提交成功后才调用 `recover` 启动执行。整个过程包在 `asyncio.shield` 里，HTTP 连接断开也不会中断。

前端这一侧同样小心。`CreationClient` 把待发送的 `generation.submit` 存进 `sessionStorage`，同一配置的重试复用原 `commandId`。响应丢失时先查回执，注释写得很直接：“丢失 HTTP 响应，不等于获准再发起一次付费请求。”

执行：每一步都先落盘

▶ 一次视频生成的执行阶段 1 / 6



1. 排队

`_run` 受全局并发 2 限制（`Semaphore(2)`）。自部署视频额外按服务地址串行，等待时间不占用并发名额和超时。

2. 提交前先写 phase=submit

这是整个设计的关键一行：进入 provider 调用**之前**先保存 `phase`。之后崩溃，看到的是“可能已经发出去”，而不是“还在排队”。

3. 拿到远端 ID 立刻保存

`submitted` 回调把 `externalTaskId` 写入工程。从此恢复时可以查询原任务，不需要重新提交。

4. 结果地址也要保存

`result_ready` 回调把下载地址写进 `.generation-results/`。下载失败时只重新下载。

5. 先存字节，再登记

完整字节先写入本地缓存，这一段在 `CancelScope(shield=True)` 内，取消信号不能打断。登记失败时重试的只是本地提交。

6. 技术检查

`_technical_check` 只证明文件能读、格式正确。Job 里同时记录 `humanReviewRequired`：内容质量仍要人来看。

恢复决策表

`recovery` 是启动恢复和手动恢复共用的唯一决策表：

依次判断	结论	行为
任务已成功	`done`	无
本地已有结果或完整缓存	`local`	只做本地登记，绝不调用模型
远端确认失败/取消/过期	`retry`	用户可以显式新建尝试
视频任务有 `externalTaskId`	`remote`	重新查询，刷新可能过期的下载地址
有已保存的结果地址	`download`	只下载，且不附带现在配置的模型凭证
`phase` 仍是 `queued`	`submit`	已证明没发出，可以提交
提交阶段收到明确的 4xx/429	`retry`	远端明确拒绝过
其他所有情况	`review`	**人工核对**，不自动做任何事

最后一行就是“未知提交”的归宿。文档的原话是“未知提交不能通过换 ID 自动重发”。

为防止两个 API 进程抢同一任务，`_run` 用按任务的文件锁认领，进程死亡时锁自动释放。

采用：成功不等于替换

结果登记时，`_register_output` 要在最新快照上重新判断是否采用。条件全部满足才替换当前内容：

- 用户没有取消；
- 当前输入算出的指纹和 Job 的指纹相同，也就是提交后用户没改过配置；
- 当前采用的结果和提交时记录的一致；

- 节点未被锁定，模型配置没换，节点上的手工素材没变；
- 策略是 `if_current`。

不满足时结果依然保存为候选，只是不上架。

取消有四种远端状态（`not_requested`、`requested`、`confirmed`、`unknown`），见 `cancel`。本地停止不代表远端已取消，也不代表不收费，界面文案也如实这样写。

> **注意 · 容易误解**
>
> - **改了服务地址、模型名或协议，任务就不再自动恢复。** `recover` 比较当前配置和快照里的地址、模型、协议，不一致就转为 `interrupted` + `review`，避免把凭证发给另一个服务。
> - **超时不等于失败。** 云端任务默认 30 分钟、自部署视频 4 小时（`_execution_timeout`），超时后状态是 `interrupted`，云端任务并没有被取消。
> - **快照里没有密钥。** `ModelSnapshot` 只记录连接事实，执行时再从 `models.json` 解析凭证。

模型接入：能力规则与协议适配

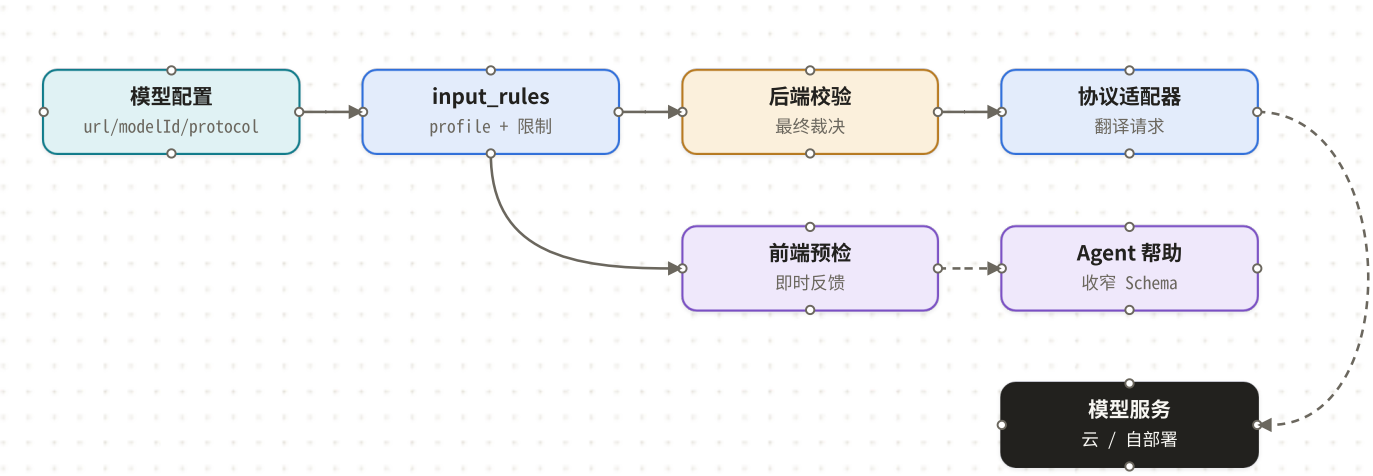
职责：让前端、Agent 和后端对“这个模型能做什么”有同一个答案，并把统一的请求翻译成各家协议。

难点：同样是“生成视频”，各家的参考数量、字节限制、时长、分辨率都不同。而且项目要求不能静默忽略不支持的参数。

注意这里有两套体系，贡献者很容易搞混：

	生成节点	工具任务
入口	`GenerationProviders`	`adapter_for`
按什么分发	`protocol` + `profile`	能力字符串，如 `image.edit`
规格来源	`input_rules`	`Capability`
实现	Volcengine、MiniMax、vLLM-Omni 等类	`ModelAdapter` 子类

一个模型配置怎样决定行为



前端预检只为了快，最终由后端决定。注释第一句就是“显式的适配器配置，不靠模型名猜测，也不在线探测”，见 `input_rules.py`。Agent 看到的命令帮助也用同一份规则收窄，见 `AgentTools.execute`。

`_adapter_rules` 是规则的中心。各 profile 的上限差别明显：

Profile	服务	视频模式	参考上限（图/视频/音频）
`seedance-2.0`	火山方舟	首帧、参考	9 / 3 / 3
`minimax-h3-official`	MiniMax 官方	首帧、参考	9 / 3 / 3
`vllm-omni-h3`	自部署 vLLM-Omni	首帧、参考、**接续**	9 / 0 / 3
`fasth3-omni`	自部署 FastH3	参考	9 / 3 / 3
`standard-video`	其他 vLLM-Omni 视频模型	首帧	1 / 0 / 0

视频规格（时长、分辨率）可以按服务配置，见 `video-model-capabilities.md`。项目还有“接续”模式：`vllm-omni-h3` 从同一服务缓存的 latent 继续上一段视频，所以不上传视频本身，这也是它的视频参考上限是 0 的原因。

配置保存在 `ModelConfigs`。`connection_fields` 限定每类模型可用的协议，`proxyMode` 让单个服务绕过系统代理。换地址时不能复用旧密钥，这条规则写在 `discover_models` 这类接口里。

工具任务：有自己台账的配方

职责：跑抠图、局部重绘、扩图、视频抠像这类一键工具，一共 21 个，定义在 `TOOLS`。

为什么不复用 Job：工具是一段配方，一次可能调用多个模型，再用 FFmpeg/Pillow 做本地处理，还可能产出多个结果。Job 的模型是“一个配置、一次调用、一个结果”，硬套就要给 GenerationSpec 加一堆假字段。文件开头的注释说的就是这个意思：配方有自己的持久台账，而不是假装成 GenerationSpec 任务，见 `creation_tool_assets.py`。

维度	生成 Job	工具任务
存在哪里	提交链（快照内）	`tool-tasks/<id>.json`
写入方式	`commit`	`write_json` + 自身状态转换
产物	ResultVersion	素材 + 画布节点
调用次数	一次	多次，记录在 `modelCalls`
能否被 Agent 调用	能	**当前不能**，见下文

`ToolSpec` 定义一个工具：接受什么来源、需要哪些能力、配方函数。配方拿到的是 `ToolContext`，只能说“编辑这张图”，从不提模型名或协议。`providers` 按配置顺序选出能提供该能力的模型。

取消同样诚实：`cancellation_state` 从每次远端调用的状态推导总体状态，存在无法确认的调用就报 `unknown`。

有些工具不需要模型，比如基于 FFmpeg 的视频放大和补帧（`_needs()` 为空）。本地媒体处理集中在 `media/processing.py`。

> **注意 · 容易误解：工具任务目前没有 Agent 入口（读代码得出）**
>
> 工具任务只通过 `/tool-tasks` HTTP 路由暴露，见 `register_tool_routes`。Agent 的业务命令表里没有对应命令，这和“能力等价”的目标还有差距。想让 Agent 用这些工具，这是一个明确的贡献方向。Agent 目前能做的替代方式是用 Shell 里的 FFmpeg 处理，再登记产物。

Agent 运行时：把外部程序请进工程

职责：让任意一个 Agent CLI 能操作 OpenDirector 工程，同时 OpenDirector 不用自己写 Agent 循环。

难点：五种程序的事件格式、认证、技能加载和权限模型都不同，而且 Agent 的 Shell 权限很大，不能让它绕过业务规则。

设计决定：项目不自己实现 Agent 循环，也不接 MCP（OpenCode 配置里 `mcp` 是空的），而是**让 Agent 在 Shell 里运行 `opendirector` 命令**。这样所有程序只需要会跑 Shell，命令的 `--help` 就是工具文档。

六个 runtime

`RUNTIMES` 登记了这些入口：

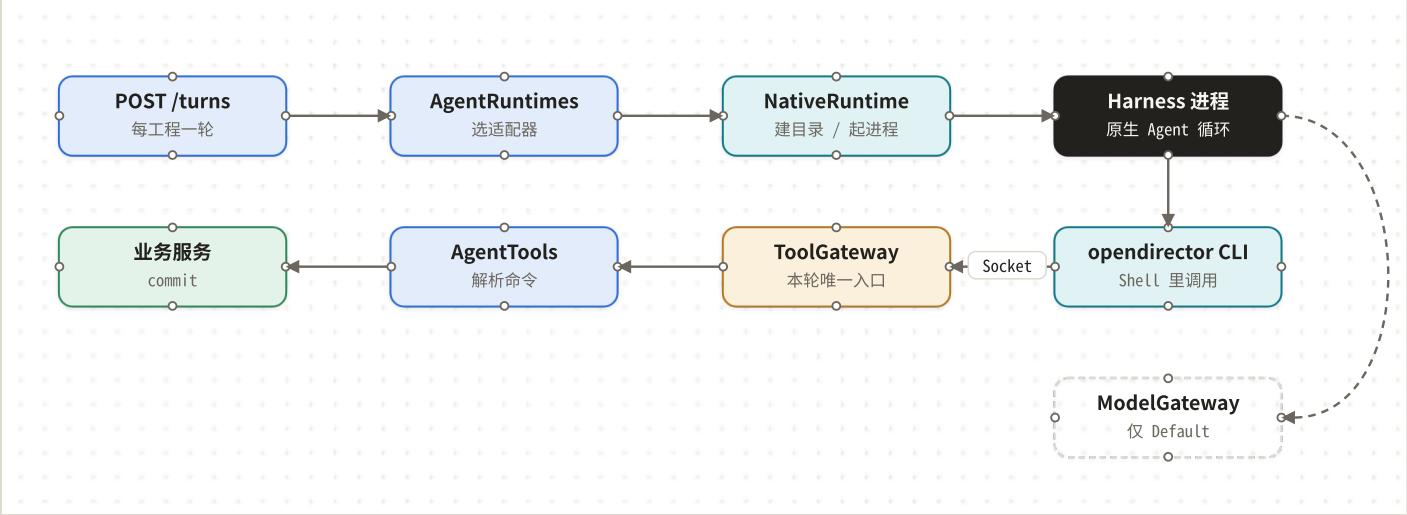
Runtime	程序	模型与认证	工具限制
Default	内置 OpenCode	OpenDirector 配置，经本地网关	Shell、读文件、已发现技能
OpenCode	PATH 中的 `opencode`	CLI 原生配置	同上
Codex	`codex`	原生登录	`workspace-write` 沙箱
Claude Code	`claude`	原生登录	仅 `Skill,Bash,Read`
JiuwenSwarm	`jiuwenswarm-process`	本机配置	stdin/stdout JSONL
DeepSeek Harness	`dsh`	CLI 原生 profile	ACP JSON-RPC

类比 · 外包施工队进场

Agent 程序像一支外包施工队，有自己的工具和干活方式，OpenDirector 不管他们怎么干。但进场有规矩：每轮发一间**独立工棚**（`agent-runs/<turn>/work`）；原材料只能看不能改；要往工地上放东西，必须走**前台窗口**（`opendirector` 命令），前台会检查单据、盖章、留回执。工棚里随便怎么折腾，只有交到前台的才算工程内容。

一轮回复怎样执行

Agent 一轮的组件关系



上面一行建立环境，下面一行是业务调用路径。适配逻辑在 `NativeRuntime.stream`，启动参数在 `command`。`ModelGateway` 只监听回环地址，内置 OpenCode 的配置里没有真实 API Key，只有本轮的临时令牌。

几个值得看的细节：

- **CLI 怎么知道自己在 Agent 里**：`environment()` 把一个薄启动脚本放到 PATH 最前面，并设置 `OPENDIRECTOR_AGENT_SOCKET`。`Client` 看到这个变量就走 Socket，否则走 HTTP，见 `environment`。
- **权限收窄**：`EXCLUDED` 去掉 8 种命令，所以 Agent 可用 32 种。比如不能创建工程，也不能直接写 `job.checkpoint`。
- **幂等**：Agent 的 `commandId` 是 `uuid5(turn_id, call_id)`。`ToolGateway.call` 在本轮内缓存调用，同 ID 不同输入直接拒绝，还用 `asyncio.shield` 防止连接取消丢掉已接受的写入。
- **帮助即文档**：`register_command` 从 Pydantic 契约自动生成子命令，所以 `Contract` 基类强制每个字段都写 `description`，缺了启动就报错。
- **运行不属于请求**：`RunEvents` 把事件留到回合结束。切换工程只断开观看连接，回来重新接入从头回放。
- **技能**：内置技能在 `packages/agent/skills/`，目前只有 `character-replacement`。工程或工作区的 `.opendirector/skills/` 也会被发现，每轮把技能复制到执行目录，见 `prepare_skills`。
- **轨迹**：原生事件写入 `agent-runs/<turn>.<runtime>.jsonl`，Default 模式还把转发的模型请求写入 `.model.jsonl`（权限 `0600`），调试 Agent 时先看这里。

注意 · 容易误解：三个边界问题

1. **Agent 的写入依赖浏览器的租约。** 一轮回复用的是发起它的浏览器 session。**实测**：在固定 commit 上，同一 session 打开另一工程后，以 `agent` 来源写原工程返回 409（“工程已关闭或会话已过期”）。所以用户切走工程时，Agent 进程继续运行、回复继续生成，但业务写入会失败。已提交的后台生成任务不受影响，因为它们走 `background` 来源。关闭页面时同样会释放租约。
2. **业务规则不是沙箱。** “不要直接改工程存储”写在系统提示里（`SYSTEM`）。OpenCode 和 Claude Code 模式下 Shell 没有操作系统沙箱，Agent 进程继承服务端环境，能读到的文件范围取决于运行用户（推断，未实测）。文档也承认“工具允许列表与操作系统沙箱提供不同约束”。不要把不可信的提示内容交给它。
3. **文档之间有矛盾。** `cli.md` 说本轮连接“受令牌保护”，而 `agent-runtime.md` 说“不使用连接令牌”。代码和后者一致：`start_server` 不校验令牌。POSIX 下靠临时目录里的 Unix Socket 隔离；Windows 下退回 `127.0.0.1` 的 TCP 端口（读代码得出，未在 Windows 运行）。

前端：画布是投影

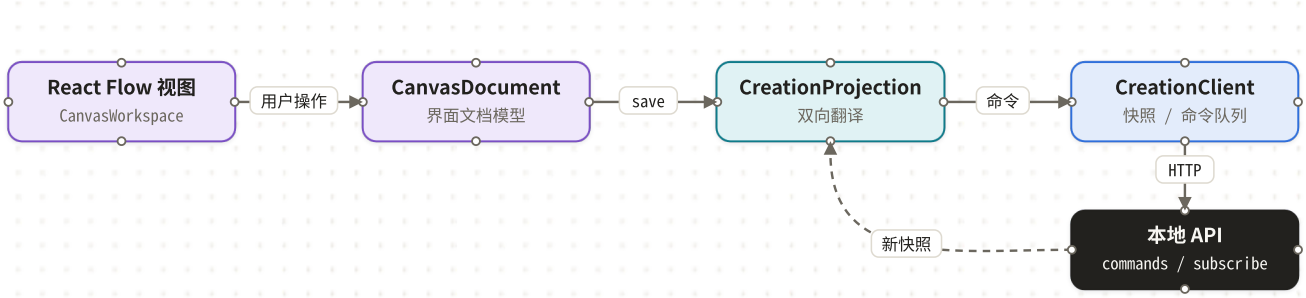
职责：提供画布、编辑器、时间线和 AI 助手界面。

难点：手感要像本地应用，拖动要流畅、输入不能被打断；但浏览器里不能有第二份可独立修改的工程。

前端是 React 19 + `@xyflow/react`，用 Vite 构建，没有 Redux 这类全局状态库。代码集中在 `src/canvas/`，`CanvasWorkspace.tsx` 单文件 6761 行，改动前要做好心理准备。

三层

浏览器里的三层



`CreationProjection` 的文件头写着“React Flow 是投影”。读取方向用 `read` 把快照变成界面文档，写入方向用 `prepare` 比较差异，翻译成类型化命令。这个文件有将近 2000 行，所有“界面改动怎样变成业务命令”的逻辑都在里面。

同步：事件只是提示

同步的设计很简单：**事件只说“变了”，内容再去查**。

`subscribe` 用 fetch 读 NDJSON 流（因为要带自定义头，不能用 `EventSource`）。收到 `cursor` 大于本地的事件就调用 `refresh` 取权威快照。服务端的 `subscribe` 每秒轮询持久化的提交链，而不是进程内的事件总线，所以断线重连和服务重启都不会丢事件。重复或乱序的事件靠比较游标自动忽略。

这样 Agent 或后台任务的提交自然会出现在画布上，前端不需要知道它们是谁做的。

`enqueue` 把命令串行排队。`CreationEditorSession` 让每个编辑弹窗持有自己的草稿和版本基线，响应未知时保留已准备好的命令。

契约由后端生成

TypeScript 类型不是手写的。`generate_contracts.py` 从 Pydantic 模型生成 `generated.ts` 和 JSON Schema，CI 用 `--check` 确认没有漂移。改了后端契约不重新生成，CI 会失败。

国际化见 `i18n.md`：中文是键的基准，英文覆盖同一组键。后端错误走 `localization.py`。

注意 · 容易误解

- **前端有一些规则和后端重复。** `rules.ts` 做预检，这是为了即时反馈，真正的判断在后端。新增规则要改后端，前端跟着改。
- `src/domain/bannerSnapshot.ts` **是遗留文件**，当前只被测试引用。见“开源许可证”一章。

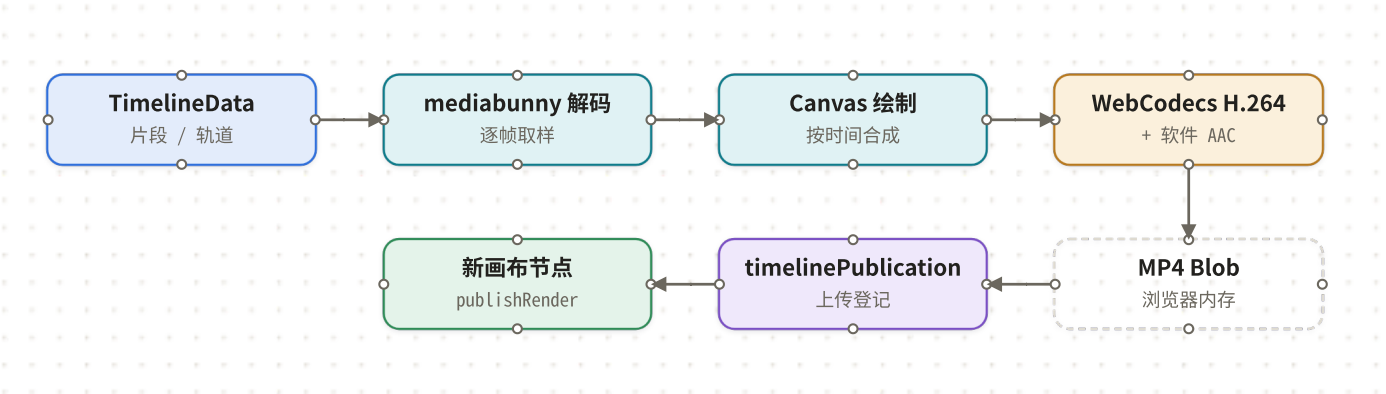
时间线与导出

职责：把素材排成时间线，导出成片或交换格式。

难点：要在没有后端渲染服务的情况下得到真实 MP4。

时间线是一种编辑文档，结构见 `TimelineData`，保存片段的起止、速度和音量。`timelineSegments` 是界面和渲染共用的语义。

MP4 在浏览器里合成



渲染入口 `renderTimelineMp4`，编码在 `encodeMp4`，AAC 是软件编码（`softwareAac.ts`）。发布用 `timelinePublication` 在渲染开始时捕获 `commandId`，渲染过程中继续编辑也不影响发布目标。

FCPXML 和素材包导出在 `timelineExport.ts`，都是纯前端逻辑。

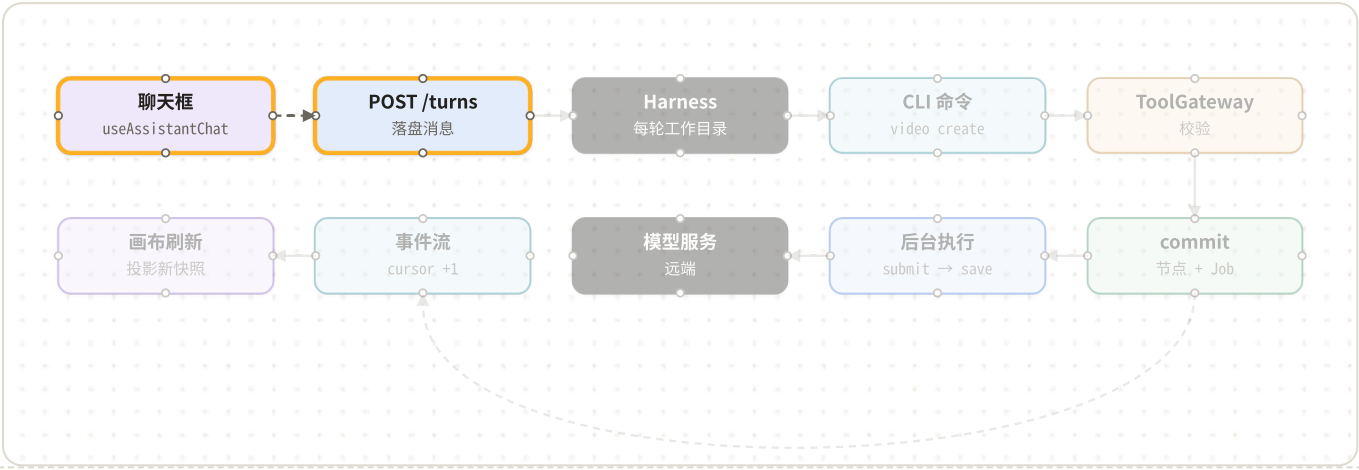
> **要点 · 这是全系统最明确的能力缺口**
>
> `timeline.publishRender` 和 `canvas.publishMedia` 在 `Agent 排除列表` 里，后端也没有渲染服务。所以 **Agent 能排时间线，但不能用时间线导出 MP4**。路线文档明确提醒不能把“可修改时间线”等同于“有后台渲染”，见 `AGENTS.md`。Agent 要出成片，目前只能在工作目录里用 FFmpeg 自己拼接，再用 `asset import` 登记。这样得到的成片和时间线文档没有关联。

其他限制来自环境：无 GPU 的无头浏览器只暴露 640x360 的 H.264 编码能力，所以 CI 用这个尺寸，1080p 导出靠人工验收，见 `validation.md`。导出期间要保持页面打开，官方推荐 Chrome 或 Edge。

端到端生命周期：一次 Agent 请求

场景：用户在右侧聊天框说“用这张图生成 5 秒视频”。全部代码都在本仓库，模型服务除外。

▶ 从一句话到画布上的视频 1 / 9



1. 接受这一轮

前端 POST `/turns`。`agent_routes` 检查每工程只有一轮、runtime 可用，把用户消息写入 `conversation.json`，引用的素材以绝对路径放进上下文，并提示“这些是引用，不是视觉观察”。

2. 启动 Harness

建立 `agent-runs/<turn>/work`，复制技能，启动 Socket 和进程。系统提示要求先运行 `opendirector --help`。

3. Agent 了解工程

Agent 跑 `opendirector project show` 和 `model list`，这些查询经 Socket 进入 `AgentTools.execute`，返回当前快照和模型规则。

4. 创建生成节点

`video create` 发出 `canvas.createGeneration`，经校验后提交，以 `agent` 来源写入。回执里直接带着下一步的提交命令，包括当前版本号。

5. 画布同步出现节点

提交产生 cursor+1 的事件，浏览器 `subscribe` 看到后刷新快照。用户这时就能看到节点，可以直接修改提示词。

6. 提交生成

`generation.submit` 进入 `prepare`，冻结输入，创建 Job，回执状态是 `accepted`，并返回 `job wait` 命令。如果用户刚改过提示词，版本检查会拒绝，Agent 必须重新读取。这就是“尊重用户修改”的实现方式。

7. 后台执行

Job 启动，走完 submit、poll、download、save。CLI 进程退出也不影响，后台写入用 `background` 来源，不需要租约。

8. 结果上架

`_register_output` 登记素材，判断采用，事件再次触发前端刷新。

9. 收尾

`job wait` 返回 `delivery`，告诉 Agent 素材已登记、不要重复发布。Agent 总结结果，回复和工具活动存入对话。用户在画布上继续编辑，下一轮 Agent 读到的就是用户改过的版本。

开源许可证

> **注意 · 这不是法律意见**
>
> 下面是对仓库内容的梳理。如果要用于商业发布，请咨询律师，尤其是本章最后一节。

各部分的许可证

部分	许可证	证据
OpenDirector 本体	Apache-2.0	`LICENSE`、`NOTICE`
`src/assets` 下的美术资源	与本体相同	`NOTICE`
`mediabunny`、`@mediabunny/aac-encoder`	MPL-2.0	`NOTICE`，未修改，从 npm 取得
`@xyflow/react`、`react`、`three`	MIT	`NOTICE`
`opencode-ai`（**二进制随 wheel 分发**）	MIT（按 lockfile）	`build_backend.py`
`streamdown`	Apache-2.0（按 lockfile）	`package.json`
`@lobehub/icons-static-svg`	MIT（按 lockfile）	`package.json`

用大白话说：Apache-2.0 允许商用、修改、闭源分发，条件是保留版权和 NOTICE、标明改动。它还包含专利授权，以及在你发起专利诉讼时终止授权的条款。MPL-2.0 是文件级的弱 copyleft：修改了 `mediabunny` 的文件才要公开这些文件的修改，只是使用不影响你自己的代码。

> **注意 · 两个可以提 PR 的小问题**
>
> - NOTICE 写的是“随本应用分发的第三方组件”，但 lockfile 显示 `opencode-ai`、`streamdown`、`@lobehub/icons-static-svg` 的运行时依赖不在清单上。构建时会复制 `LICENSE.opencode`，但只在文件存在时才复制。
> - 模型 Logo 来自图标包（`modelLogoUrls.ts`），包含 Anthropic、阿里等公司的标识。图标包的 MIT 许可证不授予这些公司的商标权。如果你发布定制版，尤其是商业版，这一点需要单独考虑。

场景表

场景	允许吗	要做什么
自己用	允许	无
给上游提 PR	允许	仓库没有 CLA 文件；按 Apache-2.0 第 5 条，贡献默认按该许可证提交
公开 fork	允许	保留 LICENSE、NOTICE，标明修改过的文件
发安装包 / wheel	允许	带上 LICENSE、NOTICE 和第三方许可证；注意 wheel 含平台二进制
对外提供服务	允许	Apache-2.0 没有网络条款。但本项目是单机设计，没有认证，先解决安全问题
闭源定制	允许	不要修改 MPL 文件，或公开这些修改

历史与一段需要知道的来源

用 Git 历史核实的时间线：

- **2026-10-26**：最早的提交。`c38cf23c` 的标题是“add reconstructed Jimeng study project”。
- **2026-11-01**：`04f0d27b` 删除研究资料和恢复工具。
- **2026-11-02**：`0b8a99e3`（#25）加入 LICENSE 和 NOTICE。提交说明写明：此前仓库没有许可证，“默认是保留所有权利”。

> **注意 · 项目早期包含对商业产品前端的反编译产物**
>
> `c38cf23c` 的 README 自述是“即梦前端学习工程”：`research/capture/` 保存了从即梦公开 CDN 采集的 JS/CSS，`recovered/` 保存 Wakaru 解包和有限反编译得到的派生模块（共 37,494 个文件，其中 JS 18,730 个），`src/` 是重新编写的 React 工程。
>
> **当前树里已经没有这些目录**，我在固定 commit 上确认过。但有三点需要知道：
>
> 1. **它们还在 Git 历史里。** 我用 `git cat-file` 确认 `c38cf23c:recovered/README.md` 仍可读取。公开 fork 这个仓库，等于连历史一起再分发。
> 2. **当前代码的来源没有完整说明。** 当时的 README 明确说 `src/` 是重写的，但也说 `bannerSnapshot.ts` “依据真实模块移植”。这个文件至今还在（`bannerSnapshot.ts`），只被测试引用。当前的 UI 布局和交互是否受原产品影响，我无法从仓库判断；一个可见的痕迹是前端路由仍用 `/ai-tool/`（`canvas_route`），和当时的目标页面路径相同。
> 3. **Apache-2.0 只能授予作者拥有的权利。** 如果某部分确实派生自第三方代码，LICENSE 声明不能解决那部分的权利。这是法律判断，我没有结论。
>
> 如果你要做商业发行，建议：向律师确认这段历史；考虑从干净的历史开始分发；删除 `bannerSnapshot.ts` 及其测试，这是低成本的清理。

贡献与定制

开发环境

需要 Python 3.11+、Node 24.22.1+、FFmpeg（含 `libx264`）。下面的命令来自 `validation.md`：

```
python3 -m venv .venv
.venv/bin/python -m pip install -e ".[test]"   # 同时运行 npm ci 和前端构建
npm exec --no -- opencode --version

opendirector dev --port 18000 --web-port 5173  # API + Vite

npm test                                       # 前端测试
.venv/bin/python -m pytest tests -q            # 后端测试
npm run build
npm run check:i18n
.venv/bin/python scripts/generate_contracts.py --check
```

调试时要注意：editable 安装的默认数据根是 checkout 目录，你的测试工程会出现在 `projects/` 下。想隔离就设 `OPENDIRECTOR_ROOT`。

Review 规则清单

来自 `AGENTS.md` 和 `ci.yml`：

- [ ] 业务规则放在领域和应用层，HTTP、UI、Agent 工具只做转换。
- [ ] 新能力同时考虑 UI 和 Agent 入口。
- [ ] 改了契约就重新生成 `generated.ts`。
- [ ] 用户可见文案同时提供 `zh-CN` 和 `en-US`，并实际切换语言看过。
- [ ] 不写旧格式迁移；不支持的格式明确拒绝。
- [ ] 验证要落到持久化对象和实际输出，“节点出现了”不算证据。
- [ ] 写明用的是模型替身还是真实服务。
- [ ] 文档随代码一起改。

提交标题用 `[bugfix] scope: ...`、`[feat]`、`[docs]`、`[test]`、`[chore]`，正文以 `## summary` 开头，见 `git-commit skill`。`.agents/skills/` 里还有 `bug-issue`、`feat-issue`，给代码 Agent 用。

测试结构

层	位置	特点
前端单元	`tests/*.test.ts`	`node --test`，无需框架
后端	`tests/test_*.py`	pytest，用 HTTP 替身和真实媒体文件
浏览器	`tests/browser/`	隔离服务 `creation_server.py` + Playwright
安装	`verify_install.py`	无 Node 环境验证 wheel

CI 在 Ubuntu 22.04 上全部运行。v0.1.0 的记录：pytest 937 项、npm test 411 项通过，但**真实模型端到端验收未执行**，见 `validation.md`。

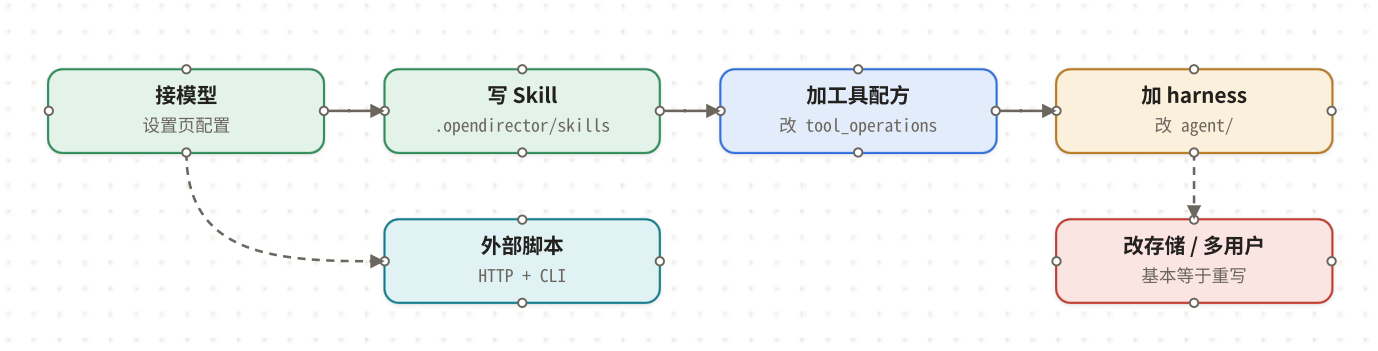
适合入手的方向

方向	难度	为什么合适
补全 NOTICE；修 `cli.md` 的令牌描述	低	本文已指出具体位置
删除 `bannerSnapshot.ts` 及其测试	低	只被测试引用
新增 `input_rules` profile	中	边界清晰，有 `tests/test_*` 可参考
新增工具配方	中	写一个 `ToolSpec` 和配方函数，再补双语词条
接入新的 harness	中	仿照 `deepseek.py`
给 Agent 开放工具任务	中高	需要设计命令和回执，补上能力等价的缺口
让 Agent 回合持有自己的租约	高	解决切换工程后写入失败的问题，要改租约模型
后台渲染	高	路线上明确缺失，但需要先有需求决策

建议先避开：拆分 `CanvasWorkspace.tsx`（改动面大，上游还在快速变化）；把存储换成数据库（和项目的核心决定相反）；给 Agent 单独写业务逻辑（违反“共用业务服务”）。

定制路线

定制的几条路：从轻到重



越往右代价越高，合并上游越难。优先考虑不改代码的方式。普通终端的 CLI 用法见 `cli.md`，注意写入时工程不能在浏览器里打开，除非传入 `--session`。

路线	做法	代价
接入自有模型	在设置页配 URL 和协议	无代码；需符合某个已知协议
写 Agent Skill	放进工程或工作区的 `.opendirector/skills/`，写工作流知识	无代码；技能内容不改变业务规则
脚本化	用 `opendirector` CLI 批量操作	浏览器打开时写入被租约拒绝
加模型协议	改 `input_rules` + 适配器 + 前端设置	要跟进上游
面向团队部署	加认证、多用户、隔离	**架构上不支持**，要改租约、存储和 Agent 权限

附录：源码索引

后端

主题	位置
应用装配、本地请求防护	`main.py`
工程 HTTP 路由	`creation.py`
助手路由、回合管理	`agent.py`
提交、恢复、历史	`creation_projects.py`
租约与全局锁	`local_projects.py`
上传暂存	`creation_uploads.py`
命令定义	`commands.py`
工程聚合校验	`project.py`
画布命令实现	`creation_canvas.py`
素材登记	`creation_assets.py`
撤销、版本、删除预览	`creation_history.py`
生成任务	`creation_generation.py`
执行与恢复决策	`generation_runtime.py`
工具任务	`tool_tasks.py`
全局素材库	`library.py`

Agent 与模型

主题	位置
runtime 目录	`runtime.py`
进程适配	`native.py`
JiuwenSwarm	`jiuwenswarm.py`
工具网关	`gateway.py`
业务工具与系统提示	`tools.py`
CLI	`cli.py`
火山方舟	`volcengine.py`
MiniMax 官方	`minimax_video.py`
vLLM-Omni	`vllm_omni.py`
错误分类	`generation_errors.py`

前端

主题	位置
画布主组件	`CanvasWorkspace.tsx`
界面文档模型	`model.ts`
投影	`creationProjection.ts`
客户端与同步	`creationClient.ts`
助手界面	`useAssistantChat.ts`
时间线编辑器	`TimelineEditor.tsx`
3D 舞台	`DirectorStage.tsx`
媒体发布	`mediaPublication.ts`
自动生成的契约	`generated.ts`

文档

主题	位置
开发指导	`AGENTS.md`
统一项目架构	`unified-project.md`
Agent 运行机制	`agent-runtime.md`
发展路线	`roadmap.md`
验证指南	`validation.md`
打包	`packaging.md`